

O que são *Stored Procedures*?

Stored Procedures são blocos de código SQL armazenados dentro do banco de dados e executados pelo próprio SGBD.

Elas funcionam de forma parecida com funções ou métodos em linguagens de programação.

No caso do MySQL, uma procedure pode:

- Receber parâmetros
- Executar consultas
- Fazer inserções, alterações e exclusões
- Utilizar estruturas condicionais
- Utilizar estruturas de repetição
- Centralizar regras de negócio

Analogia simples

Imagine um restaurante:

- O cliente faz um pedido.
- A cozinha possui uma receita pronta.
- O cozinheiro executa a receita sempre da mesma forma.

A *stored procedure* é essa “receita pronta” armazenada no banco.

Em vez de enviar vários comandos SQL separados pela aplicação, você chama apenas:

```
CALL nomeProcedure();
```

Vantagens das *Stored Procedures*

1. Reutilização de código

A lógica fica pronta no banco e pode ser reutilizada por várias aplicações.

Exemplo:

- sistema web
- aplicativo mobile
- API

Todos podem chamar a mesma procedure.

2. Melhor organização

As regras ficam centralizadas no banco.

Exemplo:

- cálculo de comissão
- fechamento mensal
- atualização de estoque

3. Segurança

É possível permitir que usuários executem procedures sem acesso direto às tabelas.

Exemplo:

```
GRANT EXECUTE ON PROCEDURE cadastrar_cliente TO usuario_app;
```

4. Redução de tráfego entre aplicação e banco

Em vez de enviar vários comandos SQL:

```
INSERT ...  
UPDATE ...  
SELECT ...
```

A aplicação envia apenas:

```
CALL processarPedido();
```

5. Melhor desempenho em algumas situações

O banco já possui o plano de execução otimizado da procedure.

Desvantagens das Stored Procedures

1. Maior complexidade

Procedures grandes podem ficar difíceis de entender e manter.

2. Dependência do SGBD

Uma procedure do MySQL normalmente não funciona no:

- PostgreSQL
- SQL Server
- Oracle

Cada banco possui sintaxe própria.

3. Dificuldade de versionamento

Controlar alterações em procedures pode ser mais difícil que versionar código Java ou PHP.

4. Mistura de responsabilidades

Muita lógica no banco pode deixar:

- manutenção difícil
- testes complicados
- arquitetura confusa

5. Debug limitado

Depurar procedures é mais difícil do que depurar código Java.

Estrutura básica no MySQL

Sintaxe geral

```
DELIMITER $$
```

```
CREATE PROCEDURE nomeProcedure()  
BEGIN
```

```
    -- comandos SQL
```

```
END $$
```

```
DELIMITER ;
```

Exemplo 1 — Procedure simples

Criando tabela

```
CREATE TABLE produto(  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(100),  
    preco DECIMAL(10,2)  
);
```

Criando procedure

```
DELIMITER $$

CREATE PROCEDURE listarProdutos()
BEGIN

    SELECT * FROM produto;

END $$

DELIMITER ;
```

Executando

```
CALL listarProdutos();
```

Exemplo 2 — Procedure com parâmetros

Cadastro de produtos

```
DELIMITER $$

CREATE PROCEDURE cadastrarProduto(
    IN p_nome VARCHAR(100),
    IN p_preco DECIMAL(10,2)
)
BEGIN

    INSERT INTO produto(nome, preco)
    VALUES(p_nome, p_preco);

END $$

DELIMITER ;
```

Executando

```
CALL cadastrarProduto('Notebook', 3500.00);
```

Tipos de parâmetros

Tipo	Descrição
IN	Entrada
OUT	Saída
INOUT	Entrada e saída

Exemplo 3 — Parâmetro OUT

Contar produtos

```
DELIMITER $$

CREATE PROCEDURE totalProdutos(
    OUT quantidade INT
)
BEGIN

    SELECT COUNT(*) INTO quantidade
    FROM produto;

END $$

DELIMITER ;
```

Executando

```
CALL totalProdutos(@total);
```

```
SELECT @total;
```

Estruturas condicionais

IF

```
DELIMITER $$

CREATE PROCEDURE verificarPreco(
    IN p_preco DECIMAL(10,2)
)
BEGIN

    IF p_preco >= 1000 THEN
        SELECT 'Produto caro';
    ELSE
        SELECT 'Produto barato';
    END IF;

END $$

DELIMITER ;
```

Executando

```
CALL verificarPreco(2500);
```

Estruturas de repetição

WHILE

```
DELIMITER $$

CREATE PROCEDURE contar()
BEGIN

    DECLARE i INT DEFAULT 1;
```

```
WHILE i <= 5 DO

    SELECT i;

    SET i = i + 1;

END WHILE;

END $$

DELIMITER ;
```

Resultado

```
1
2
3
4
5
```

Variáveis locais

```
DECLARE total DECIMAL(10,2);
```

Atribuição

```
SET total = 100;
```

SELECT INTO

Permite armazenar resultado em variável.

```
SELECT preco
INTO valor
```



```
FROM produto
WHERE id = 1;
```

Exemplo completo — Controle de estoque

Tabela

```
CREATE TABLE estoque(
    id INT PRIMARY KEY AUTO_INCREMENT,
    produto VARCHAR(100),
    quantidade INT
);
```

Procedure

```
DELIMITER $$
```

```
CREATE PROCEDURE adicionarEstoque(
    IN p_produto VARCHAR(100),
    IN p_quantidade INT
)
BEGIN

    DECLARE qtdAtual INT;

    SELECT quantidade
    INTO qtdAtual
    FROM estoque
    WHERE produto = p_produto;

    IF qtdAtual IS NULL THEN

        INSERT INTO estoque(produto, quantidade)
        VALUES(p_produto, p_quantidade);

    ELSE
```

```
        UPDATE estoque
        SET quantidade = quantidade + p_quantidade
        WHERE produto = p_produto;

    END IF;

END $$

DELIMITER ;
```

Executando

```
CALL adicionarEstoque('Mouse', 10);
```

Como listar procedures no MySQL

```
SHOW PROCEDURE STATUS;
```

Como visualizar código da procedure

```
SHOW CREATE PROCEDURE listarProdutos;
```

Como remover procedure

```
DROP PROCEDURE nomeProcedure;
```

Quando usar Stored Procedures?

Bons cenários

- Regras complexas no banco

- Processamentos em lote
- Rotinas administrativas
- Auditoria
- Fechamento financeiro
- Controle de estoque

Cenários onde talvez não sejam ideais

- Sistemas muito orientados a microsserviços
- Aplicações com regras altamente dinâmicas
- Projetos que precisam trocar de SGBD facilmente

Comparação rápida

Característica	Procedure	SQL comum
Reutilização	Alta	Baixa
Performance	Boa	Média
Organização	Centralizada	Espalhada
Portabilidade	Baixa	Média
Complexidade	Alta	Baixa

Exemplo prático de uso em Java

Chamando procedure usando JDBC:

```
CallableStatement cs =  
    connection.prepareCall("{call listarProdutos()}");  
  
ResultSet rs = cs.executeQuery();  
  
while(rs.next()){  
    System.out.println(rs.getString("nome"));  
}
```

Resumo

As *stored procedures* permitem:

- encapsular lógica SQL
- reutilizar código
- melhorar segurança
- reduzir tráfego entre aplicação e banco

Por outro lado:

- aumentam acoplamento ao banco
- dificultam manutenção em excesso
- possuem sintaxe específica do SGBD

No MySQL, elas são muito utilizadas em:

- ERPs
- sistemas financeiros
- controle de estoque
- rotinas administrativas
- integrações corporativas